

核心代码附录排版测试

摘要

本文件验证核心 Python 程序完整导出、中文字符串、长行折行与跨页排版。它仅用于导出工具链测试，不是论文。

关键词：代码；附录；排版测试

A 完整核心程序

以下程序从当前核心源文件复制，未作内容修改。

```

1  """2026 CUMCM C problem: shared data, forecasting, LP, and causal simulation
   core."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  import os
7  from pathlib import Path
8  from typing import Any
9
10 import numpy as np
11 import pandas as pd
12 from scipy.optimize import linprog
13 from scipy.sparse import csr_matrix, lil_matrix
14 os.environ.setdefault("LOKY_MAX_CPU_COUNT", "4")
15
16 from sklearn.cluster import KMeans
17
18
19 DT_HOURS = 1.0 / 6.0
20 N_SLOTS = 144
21 ETA_C = 0.90
22 ETA_D = 0.90
23 SOC_MIN = 1_200.0
24 SOC_MAX = 10_800.0
25 SOC_INITIAL = 6_000.0
26 POWER_LIMIT_KW = 5_000.0
27 ENERGY_LIMIT = POWER_LIMIT_KW * DT_HOURS
28 SEED = 2026
29 DEFAULT_ALPHA = 0.80
30 ISSUES = (0, 6, 12, 18)
31
32
33 @dataclass(frozen=True)
34 class Battery:
35     capacity_kwh: float = 12000.0
36     power_kw: float = POWER_LIMIT_KW
37     eta_c: float = ETA_C
38     eta_d: float = ETA_D
39
40     @property
41     def minimum(self) -> float:

```

```

42         return 0.1 * self.capacity_kwh
43
44     @property
45     def maximum(self) -> float:
46         return 0.9 * self.capacity_kwh
47
48     @property
49     def initial(self) -> float:
50         return 0.5 * self.capacity_kwh
51
52     @property
53     def slot_limit(self) -> float:
54         return self.power_kw * DT_HOURS
55
56
57 DEFAULT_BATTERY = Battery()
58
59
60 @dataclass(frozen=True)
61 class Inputs:
62     dates: pd.DatetimeIndex
63     typical_time: list[Any]
64     typical_price: np.ndarray
65     typical_load_kw: np.ndarray
66     typical_pv_kw: np.ndarray
67     actual_load_kw: np.ndarray
68     actual_pv_kw: np.ndarray
69     variable_price: np.ndarray
70     pv_forecast_hourly: np.ndarray
71
72
73 def load_inputs(project_root: str | Path) -> Inputs:
74     root = Path(project_root)
75     attachment = root / "data" / "附件"
76
77     a1 = pd.read_excel(attachment / "附件 1.xlsx", sheet_name=0, header=0)
78     if a1.shape != (N_SLOTS, 4):
79         raise ValueError(f"附件 1 维度异常: {a1.shape}")
80
81     load_df = pd.read_excel(attachment / "附件2.xlsx", sheet_name="小区负载",
82                             index_col=0, header=0)
83     pv_df = pd.read_excel(attachment / "附件2.xlsx",
84                           sheet_name="光伏发电实际功率", index_col=0, header=0)
85     price_df = pd.read_excel(attachment / "附件4.xlsx", sheet_name=0,
86                              index_col=0, header=0)

```

```

84     def clock_minute(value):
85         if hasattr(value, "hour"):
86             minute = value.hour * 60 + value.minute
87         else:
88             clock = str(value).split("+")[0].split(":")
89             minute = int(clock[0]) * 60 + int(clock[1])
90         return 1440 if minute == 0 else minute
91     expected_slots = list(range(10, 1441, 10))
92     for labels in (a1.iloc[:, 0], load_df.columns, pv_df.columns,
93                   price_df.columns):
94         if [clock_minute(v) for v in labels] != expected_slots:
95             raise ValueError(" 附件时间列不是有序的 00:10 至 24:00 端点")
96     dates = pd.DatetimeIndex(pd.to_datetime(load_df.index))
97     if load_df.shape != (365, N_SLOTS) or pv_df.shape != load_df.shape:
98         raise ValueError(" 附件 2 应为 365×144 的负荷和光伏矩阵")
99     if price_df.shape != load_df.shape:
100         raise ValueError(" 附件 4 应与附件 2 严格同形")
101     if not dates.equals(pd.DatetimeIndex(pd.to_datetime(pv_df.index))):
102         raise ValueError(" 附件 2 两个工作表日期不一致")
103     if not dates.equals(pd.DatetimeIndex(pd.to_datetime(price_df.index))):
104         raise ValueError(" 附件 4 日期与附件 2 不一致")
105
106     a3 = pd.read_excel(attachment / " 附件 3.xlsx", sheet_name=0, header=0)
107     if a3.shape != (1460, 26) or list(a3.columns[2:]) != [f"预报{h}小时" for h
108     in range(1, 25)]:
109         raise ValueError(" 附件 3 维度或提前期表头异常")
110     forecast_dates = pd.to_datetime(a3.iloc[:, 0].replace("", np.nan).ffill())
111     forecast = np.full((len(dates), len(ISSUES), 24), np.nan, dtype=float)
112     date_pos = {d.normalize(): i for i, d in enumerate(dates)}
113     issue_pos = {h: i for i, h in enumerate(ISSUES)}
114     for row_index, row in enumerate(a3.itertuples(index=False, name=None)):
115         d = pd.Timestamp(forecast_dates.iloc[row_index]).normalize()
116         issue_value = row[1]
117         issue_hour = int(issue_value.hour) if hasattr(issue_value, "hour") else
118         int(str(issue_value).split(":")[0])
119         if d not in date_pos or issue_hour not in issue_pos:
120             raise ValueError(" 附件 3 含未定义日期或发布时刻")
121         target = forecast[date_pos[d], issue_pos[issue_hour]]
122         if np.isfinite(target).any():
123             raise ValueError(" 附件 3 日期与发布时刻重复")
124         target[:] = np.asarray(row[2:26], dtype=float)
125     if np.isnan(forecast).any():
126         raise ValueError(" 附件 3 存在未能映射的日期、发布时刻或预测值")
127
128     result = Inputs(

```

```

126         dates=dates,
127         typical_time=a1.iloc[:, 0].tolist(),
128         typical_price=a1.iloc[:, 1].to_numpy(dtype=float),
129         typical_load_kw=a1.iloc[:, 2].to_numpy(dtype=float),
130         typical_pv_kw=a1.iloc[:, 3].to_numpy(dtype=float),
131         actual_load_kw=load_df.to_numpy(dtype=float),
132         actual_pv_kw=pv_df.to_numpy(dtype=float),
133         variable_price=price_df.to_numpy(dtype=float),
134         pv_forecast_hourly=forecast,
135     )
136     _validate_inputs(result)
137     return result
138
139
140 def _validate_inputs(data: Inputs) -> None:
141     arrays = (
142         data.typical_price,
143         data.typical_load_kw,
144         data.typical_pv_kw,
145         data.actual_load_kw,
146         data.actual_pv_kw,
147         data.variable_price,
148         data.pv_forecast_hourly,
149     )
150     if any(not np.isfinite(a).all() for a in arrays):
151         raise ValueError(" 输入包含 NaN 或无穷值")
152     if any((a < 0).any() for a in arrays):
153         raise ValueError(" 输入包含负的电价、负荷或光伏值")
154     expected = pd.date_range("2025-01-01", "2025-12-31", freq="D")
155     if not data.dates.equals(expected):
156         raise ValueError(" 附件日期并非 2025 完整连续自然年")
157
158
159 def build_analog_forecasts(
160     actual_kw: np.ndarray,
161     typical_kw: np.ndarray,
162     *,
163     weekday_weight: bool,
164     max_history: int = 90,
165     decay_days: float = 60.0,
166 ) -> tuple[np.ndarray, np.ndarray]:
167     """Strict rolling-origin analog forecast and its realized residuals."""
168     n_days, n_slots = actual_kw.shape
169     pred = np.empty_like(actual_kw, dtype=float)
170     for d in range(n_days):
171         start = max(0, d - max_history)

```

```

172         idx = np.arange(start, d)
173         if idx.size == 0:
174             pred[d] = typical_kw
175             continue
176         ages = d - idx
177         weights = np.exp(-ages / decay_days)
178         if weekday_weight:
179             weights *= np.where(idx % 7 == d % 7, 2.0, 1.0)
180         hist_mean = np.average(actual_kw[idx], axis=0, weights=weights)
181         shrink = min(1.0, idx.size / 21.0)
182         pred[d] = shrink * hist_mean + (1.0 - shrink) * typical_kw
183     pred = np.maximum(pred, 0.0)
184     return pred, actual_kw - pred
185
186
187 def _cluster_residuals(
188     load_residual_kw: np.ndarray,
189     pv_residual_kw: np.ndarray,
190     *,
191     max_scenarios: int = 9,
192 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
193     n = load_residual_kw.shape[0]
194     if n == 0:
195         zeros = np.zeros((1, load_residual_kw.shape[1]), dtype=float)
196         return zeros, zeros.copy(), np.ones(1)
197     raw = np.concatenate([load_residual_kw, pv_residual_kw], axis=1)
198     scale = np.std(raw, axis=0)
199     scale[scale < 1e-9] = 1.0
200     k = min(max_scenarios, n, np.unique(raw / scale, axis=0).shape[0])
201     if k == 1:
202         labels = np.zeros(n, dtype=int)
203     else:
204         labels = KMeans(n_clusters=k, random_state=SEED,
205             n_init=10).fit_predict(raw / scale)
206     nonempty = np.unique(labels)
207     centers_l = np.stack([load_residual_kw[labels == i].mean(axis=0) for i in
208         nonempty])
209     centers_v = np.stack([pv_residual_kw[labels == i].mean(axis=0) for i in
210         nonempty])
211     weights = np.asarray([(labels == i).mean() for i in nonempty], dtype=float)
212     return centers_l, centers_v, weights
213
214 def weighted_quantile_columns(values: np.ndarray, weights: np.ndarray,
215     quantile: float) -> np.ndarray:

```

```

213     if values.ndim != 2 or values.shape[0] != weights.size:
214         raise ValueError(" 分位数输入维度不一致")
215     if not 0 <= quantile <= 1 or not np.isfinite(values).all() or not
np.isfinite(weights).all() or np.any(weights < 0) or weights.sum() <= 0:
216         raise ValueError(" 分位数或概率非法")
217     result = np.empty(values.shape[1], dtype=float)
218     for t in range(values.shape[1]):
219         order = np.argsort(values[:, t], kind="mergesort")
220         v = values[order, t]
221         w = weights[order]
222         cdf = np.cumsum(w) / np.sum(w)
223         result[t] = v[min(np.searchsorted(cdf, quantile, side="left"), len(v) -
1)]
224     return result
225
226
227 def safe_analog_trajectory(
228     day: int,
229     load_point_kw: np.ndarray,
230     pv_point_kw: np.ndarray,
231     load_residual_kw: np.ndarray,
232     pv_residual_kw: np.ndarray,
233     *,
234     alpha: float = DEFAULT_ALPHA,
235     history_days: int = 60,
236     max_scenarios: int = 9,
237 ) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
238     start = max(0, day - history_days)
239     c_l, c_v, weights = _cluster_residuals(
240         load_residual_kw[start:day], pv_residual_kw[start:day],
241         max_scenarios=max_scenarios
242     )
243     l_scen = np.maximum(0.0, load_point_kw[None, :] + c_l)
244     v_scen = np.maximum(0.0, pv_point_kw[None, :] + c_v)
245     net_q = weighted_quantile_columns(l_scen - v_scen, weights, alpha)
246     pv_low = weighted_quantile_columns(v_scen, weights, 1.0 - alpha)
247     safe_pv = np.maximum.reduce([pv_low, -net_q, np.zeros_like(net_q)])
248     safe_load = safe_pv + net_q
249     return safe_load, safe_pv, {
250         "alpha": alpha,
251         "history_start": start,
252         "history_end_exclusive": day,
253         "scenario_count": int(weights.size),
254         "scenario_weights": weights.tolist(),
255     }

```

```

255
256
257 def _block_slices(n: int, adjusted: bool) -> dict[str, slice]:
258     pos = 0
259     blocks: dict[str, slice] = {}
260     for name, size in (("grid", n), ("charge", n), ("discharge", n), ("soc", n
+ 1), ("pv_used", n)):
261         blocks[name] = slice(pos, pos + size)
262         pos += size
263     if adjusted:
264         blocks["up"] = slice(pos, pos + n)
265         pos += n
266         blocks["down"] = slice(pos, pos + n)
267         pos += n
268     blocks["all"] = slice(0, pos)
269     return blocks
270
271
272 def solve_schedule(
273     load_kw: np.ndarray,
274     pv_kw: np.ndarray,
275     price: np.ndarray,
276     soc0: float,
277     *,
278     terminal_cycle: bool = False,
279     terminal_value: float = 0.0,
280     base_plan_kwh: np.ndarray | None = None,
281     battery: Battery = DEFAULT_BATTERY,
282 ) -> dict[str, Any]:
283     """Solve a deterministic LP for an initial or adjusted contract
trajectory."""
284     load_kw = np.asarray(load_kw, dtype=float)
285     pv_kw = np.asarray(pv_kw, dtype=float)
286     price = np.asarray(price, dtype=float)
287     n = load_kw.size
288     if pv_kw.shape != (n,) or price.shape != (n,):
289         raise ValueError(" 负荷、光伏、电价长度不一致")
290     if not (battery.minimum - 1e-9 <= soc0 <= battery.maximum + 1e-9):
291         raise ValueError(f" 初始 SOC 越界: {soc0}")
292     adjusted = base_plan_kwh is not None
293     if adjusted:
294         base_plan_kwh = np.asarray(base_plan_kwh, dtype=float)
295         if base_plan_kwh.shape != (n,):
296             raise ValueError(" 原计划与调整时域长度不一致")
297     b = _block_slices(n, adjusted)

```

```

298     m = b["all"].stop
299     c = np.zeros(m, dtype=float)
300     if adjusted:
301         c[b["up"]] = 1.5 * price
302         c[b["down"]] = -0.5 * price
303         constant_cost = float(np.dot(price, base_plan_kwh))
304     else:
305         c[b["grid"]] = price
306         constant_cost = 0.0
307     c[b["soc"].stop - 1] = -float(terminal_value)
308
309     rows = 2 * n + (n if adjusted else 0)
310     aeq = lil_matrix((rows, m), dtype=float)
311     beq = np.zeros(rows, dtype=float)
312     load_energy = load_kw * DT_HOURS
313     pv_energy = pv_kw * DT_HOURS
314     for t in range(n):
315         aeq[t, b["grid"].start + t] = 1.0
316         aeq[t, b["charge"].start + t] = -1.0
317         aeq[t, b["discharge"].start + t] = 1.0
318         aeq[t, b["pv_used"].start + t] = 1.0
319         beq[t] = load_energy[t]
320
321         row = n + t
322         aeq[row, b["soc"].start + t] = -1.0
323         aeq[row, b["soc"].start + t + 1] = 1.0
324         aeq[row, b["charge"].start + t] = -battery.eta_c
325         aeq[row, b["discharge"].start + t] = 1.0 / battery.eta_d
326         if adjusted:
327             row = 2 * n + t
328             aeq[row, b["grid"].start + t] = 1.0
329             aeq[row, b["up"].start + t] = -1.0
330             aeq[row, b["down"].start + t] = 1.0
331             beq[row] = base_plan_kwh[t]
332
333     bounds: list[tuple[float | None, float | None]] = []
334     bounds.extend([(0.0, None)] * n)
335     bounds.extend([(0.0, battery.slot_limit)] * n)
336     bounds.extend([(0.0, battery.slot_limit)] * n)
337     soc_bounds = [(battery.minimum, battery.maximum)] * (n + 1)
338     soc_bounds[0] = (soc0, soc0)
339     if terminal_cycle:
340         soc_bounds[-1] = (soc0, soc0)
341     bounds.extend(soc_bounds)
342     bounds.extend([(0.0, float(v)) for v in pv_energy])
343     if adjusted:

```

```

344     bounds.extend([(0.0, None)] * n)
345     bounds.extend([(0.0, float(v)) for v in base_plan_kwh])
346
347     res = linprog(c, A_eq=aeq.tocsr(), b_eq=beq, bounds=bounds, method="highs")
348     if not res.success:
349         raise RuntimeError(f"HiGHS求解失败: status={res.status}, {res.message}")
350
351     economic_optimum = float(res.fun)
352     tie = np.zeros(m, dtype=float)
353     tie[b["charge"]] = 1.0
354     tie[b["discharge"]] = 1.0
355     aub = csr_matrix(c.reshape(1, -1))
356     tol = 1e-8 * max(1.0, abs(economic_optimum))
357     res2 = linprog(
358         tie,
359         A_ub=aub,
360         b_ub=np.asarray([economic_optimum + tol]),
361         A_eq=aeq.tocsr(),
362         b_eq=beq,
363         bounds=bounds,
364         method="highs",
365     )
366     if not res2.success:
367         raise RuntimeError(f"吞吐次级 LP 失败: {res2.status}, {res2.message}")
368     x = res2.x
369     balance_residual = aeq.tocsr() @ x - beq
370     result = {
371         "grid_kwh": x[b["grid"]].copy(),
372         "charge_kwh": x[b["charge"]].copy(),
373         "discharge_kwh": x[b["discharge"]].copy(),
374         "soc_kwh": x[b["soc"]].copy(),
375         "pv_used_kwh": x[b["pv_used"]].copy(),
376         "solver_objective": float(np.dot(c, x) + constant_cost + terminal_value
377             * battery.minimum),
378         "cash_contract_cost": float(
379             np.dot(price, x[b["grid"]])
380             if not adjusted
381             else constant_cost
382             + np.dot(1.5 * price, x[b["up"]])
383             - np.dot(0.5 * price, x[b["down"]])
384         ),
385         "terminal_value_credit": float(terminal_value * (x[b["soc"].stop - 1] -
386             battery.minimum)),
387         "max_eq_residual": float(np.max(np.abs(balance_residual))),
388         "simultaneous_flow_max": float(np.max(np.minimum(x[b["charge"]],
389             x[b["discharge"]]))),

```

```

387         "status": "optimal",
388     }
389     if adjusted:
390         result["up_kwh"] = x[b["up"]].copy()
391         result["down_kwh"] = x[b["down"]].copy()
392     return result
393
394
395 def causal_dispatch(
396     contract_kwh: np.ndarray,
397     actual_load_kw: np.ndarray,
398     actual_pv_kw: np.ndarray,
399     price: np.ndarray,
400     soc0: float,
401     *,
402     base_plan_kwh: np.ndarray | None = None,
403     battery: Battery = DEFAULT_BATTERY,
404     emergency_multiplier: float = 5.0,
405 ) -> dict[str, Any]:
406     """Greedy feasible controller using current observation only."""
407     q = np.asarray(contract_kwh, dtype=float)
408     load_e = np.asarray(actual_load_kw, dtype=float) * DT_HOURS
409     pv_e = np.asarray(actual_pv_kw, dtype=float) * DT_HOURS
410     price = np.asarray(price, dtype=float)
411     n = q.size
412     if load_e.shape != (n,) or pv_e.shape != (n,) or price.shape != (n,):
413         raise ValueError(" 因果控制输入长度不一致")
414     charge = np.zeros(n)
415     discharge = np.zeros(n)
416     emergency = np.zeros(n)
417     unused = np.zeros(n)
418     curtailed = np.zeros(n)
419     pv_used = np.zeros(n)
420     soc = np.empty(n + 1)
421     soc[0] = soc0
422     for t in range(n):
423         available = q[t] + pv_e[t] - load_e[t]
424         if available >= 0.0:
425             charge[t] = min(available, battery.slot_limit, max(0.0,
426                 (battery.maximum - soc[t]) / battery.eta_c))
427             surplus = max(0.0, available - charge[t])
428             unused[t] = min(q[t], surplus)
429             curtailed[t] = max(0.0, surplus - unused[t])
430             pv_used[t] = pv_e[t] - curtailed[t]
431             soc[t + 1] = soc[t] + battery.eta_c * charge[t]

```

```

431         else:
432             need = -available
433             discharge[t] = min(need, battery.slot_limit, max(0.0, (soc[t] -
434                 battery.minimum) * battery.eta_d))
435             emergency[t] = max(0.0, need - discharge[t])
436             pv_used[t] = pv_e[t]
437             soc[t + 1] = soc[t] - discharge[t] / battery.eta_d
438         residual = q - unused + pv_used + discharge + emergency - load_e - charge
439         if base_plan_kwh is None:
440             settlement = price * q
441             alt_settlement = settlement.copy()
442         else:
443             p = np.asarray(base_plan_kwh, dtype=float)
444             if p.shape != (n,):
445                 raise ValueError(" 原计划与最终合同长度不一致")
446             settlement = price * np.minimum(p, q) + 1.5 * price * np.maximum(q - p,
447                 0.0) + 0.5 * price * np.maximum(p - q, 0.0)
448             alt_settlement = price * p + 1.5 * price * np.maximum(q - p, 0.0) + 0.5
449                 * price * np.maximum(p - q, 0.0)
450         emergency_cost = emergency_multiplier * price * emergency
451         return {
452             "charge_kwh": charge,
453             "discharge_kwh": discharge,
454             "soc_kwh": soc,
455             "emergency_kwh": emergency,
456             "unused_contract_kwh": unused,
457             "pv_curtailed_kwh": curtailed,
458             "pv_used_kwh": pv_used,
459             "settlement_cost": settlement,
460             "alternative_settlement_cost": alt_settlement,
461             "emergency_cost": emergency_cost,
462             "cash_cost": settlement + emergency_cost,
463             "alternative_cash_cost": alt_settlement + emergency_cost,
464             "max_balance_residual": float(np.max(np.abs(residual))),
465         }
466
467
468 def terminal_value(price: np.ndarray, battery: Battery = DEFAULT_BATTERY) ->
469     float:
470     return float(battery.eta_d * np.median(np.asarray(price, dtype=float)))
471
472
473 def predict_price(data: Inputs, day: int, issue_hour: int = 0) -> np.ndarray:
474     """Only historical days and the already-realized prefix are readable."""
475     base = np.median(data.variable_price[max(0, day - 30):day], axis=0) if day
476     else data.typical_price.copy()

```

```

472     start = issue_hour * 6
473     if start:
474         observed = slice(max(0, start - 18), start)
475         bias = float(np.mean(data.variable_price[day, observed] -
476                               base[observed]))
476         tau = np.arange(1, N_SLOTS - start + 1) / 6.0
477         return np.maximum(0.0001, base[start:] + bias * np.exp(-tau / 3.0))
478     return np.maximum(0.0001, base)
479
480
481 def run_q1(data: Inputs, battery: Battery = DEFAULT_BATTERY) -> dict[str, Any]:
482     return solve_schedule(
483         data.typical_load_kw,
484         data.typical_pv_kw,
485         data.typical_price,
486         battery.initial,
487         terminal_cycle=True,
488         battery=battery,
489     )
490
491
492 def run_q2_day(
493     data: Inputs,
494     day: int,
495     soc0: float,
496     load_point: np.ndarray,
497     pv_point: np.ndarray,
498     load_residual: np.ndarray,
499     pv_residual: np.ndarray,
500     *,
501     variable_price: bool,
502     alpha: float = DEFAULT_ALPHA,
503     safe_pair: tuple[np.ndarray, np.ndarray, dict[str, Any]] | None = None,
504     battery: Battery = DEFAULT_BATTERY,
505     emergency_multiplier: float = 5.0,
506     history_days: int = 60,
507     max_scenarios: int = 9,
508 ) -> dict[str, Any]:
509     price = predict_price(data, day) if variable_price else data.typical_price
510     settlement_price = data.variable_price[day] if variable_price else
511     data.typical_price
512     if safe_pair is None:
513         safe_l, safe_v, meta = safe_analog_trajectory(
514             day, load_point[day], pv_point[day], load_residual, pv_residual,
515             alpha=alpha, history_days=history_days, max_scenarios=max_scenarios

```

```

514         )
515     else:
516         safe_l, safe_v, meta = safe_pair
517     plan = solve_schedule(
518         safe_l,
519         safe_v,
520         price,
521         soc0,
522         terminal_value=terminal_value(price, battery),
523         battery=battery,
524     )
525     actual = causal_dispatch(
526         plan["grid_kwh"], data.actual_load_kw[day], data.actual_pv_kw[day],
527         settlement_price, soc0, battery=battery,
528         emergency_multiplier=emergency_multiplier
529     )
530     return {"plan": plan, "actual": actual, "forecast_meta": meta,
531            "safe_load_kw": safe_l, "safe_pv_kw": safe_v, "planning_price": price}
532
533 def official_pv_residuals(data: Inputs) -> np.ndarray:
534     """Actual power at each forecasted clock hour minus that point forecast."""
535     n_days = len(data.dates)
536     flat = data.actual_pv_kw.reshape(-1)
537     residual = np.full_like(data.pv_forecast_hourly, np.nan, dtype=float)
538     for d in range(n_days):
539         for i, issue in enumerate(ISSUES):
540             start = d * N_SLOTS + issue * 6
541             for h in range(24):
542                 # Column h is the point forecast at issue+(h+1) hours. The
543                 # 10-minute source rows are interval endpoints, hence -1.
544                 endpoint = start + (h + 1) * 6 - 1
545                 if endpoint < flat.size:
546                     residual[d, i, h] = flat[endpoint] -
547                     data.pv_forecast_hourly[d, i, h]
548     return residual
549
550 def interpolate_hourly_points(anchor: float, hourly_points: np.ndarray,
551                                n_slots: int) -> np.ndarray:
552     """Linearly interpolate clock-hour point values onto 10-minute
553     endpoints."""
554     values = np.concatenate(([float(anchor)], np.asarray(hourly_points,
555                                                             dtype=float)))
556     x_hour = np.arange(values.size, dtype=float)

```

```

552     x_slot = np.arange(1, n_slots + 1, dtype=float) / 6.0
553     return np.interp(x_slot, x_hour, values)
554
555
556 def safe_update_trajectory(
557     data: Inputs,
558     day: int,
559     issue_index: int,
560     load_point_day: np.ndarray,
561     load_residual: np.ndarray,
562     pv_issue_residual: np.ndarray,
563     *,
564     alpha: float = DEFAULT_ALPHA,
565     history_days: int = 60,
566     max_scenarios: int = 9,
567 ) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
568     issue = ISSUES[issue_index]
569     start_slot = issue * 6
570     n = N_SLOTS - start_slot
571     hours = int(np.ceil(n / 6))
572     raw = data.pv_forecast_hourly[day, issue_index, :hours]
573     hist_start = max(0, day - history_days)
574     hist = pv_issue_residual[hist_start:day, issue_index, :hours]
575     valid_rows = np.all(np.isfinite(hist), axis=1) if hist.size else
    np.zeros(0, dtype=bool)
576     hist = hist[valid_rows]
577     bias = np.nanmean(hist, axis=0) if hist.shape[0] else np.zeros(hours)
578     if issue > 0:
579         anchor = float(data.actual_pv_kw[day, start_slot - 1])
580     elif day > 0:
581         anchor = float(data.actual_pv_kw[day - 1, -1])
582     else:
583         anchor = 0.0
584     pv_point = interpolate_hourly_points(anchor, np.maximum(0.0, raw + bias),
    n)
585     l_point = load_point_day[start_slot:]
586
587     hist_ids = np.arange(hist_start, day)[valid_rows]
588     if hist_ids.size:
589         r_l = load_residual[hist_ids, start_slot:]
590         centered_hourly = hist - bias[None, :]
591         r_v = np.vstack(
592             [interpolate_hourly_points(0.0, row, n) for row in centered_hourly]
593         )
594     else:

```

```

595         r_l = np.empty((0, n))
596         r_v = np.empty((0, n))
597     c_l, c_v, weights = _cluster_residuals(r_l, r_v,
max_scenarios=max_scenarios)
598     l_scen = np.maximum(0.0, l_point[None, :] + c_l)
599     v_scen = np.maximum(0.0, pv_point[None, :] + c_v)
600     net_q = weighted_quantile_columns(l_scen - v_scen, weights, alpha)
601     pv_low = weighted_quantile_columns(v_scen, weights, 1.0 - alpha)
602     safe_v = np.maximum.reduce([pv_low, -net_q, np.zeros_like(net_q)])
603     safe_l = safe_v + net_q
604     return safe_l, safe_v, {
605         "decision_day": int(day),
606         "issue_hour": int(issue),
607         "history_start": int(hist_start),
608         "history_end_exclusive": int(day),
609         "scenario_count": int(weights.size),
610         "alpha": float(alpha),
611         "scenario_weights": weights.tolist(),
612         "pv_point_kw": pv_point.tolist(),
613         "anchor_kw": anchor,
614         "bias_kw": bias.tolist(),
615     }
616
617
618 def nowcast_trajectory(
619     data: Inputs, day: int, event: int,
620     source: tuple[np.ndarray, np.ndarray, dict[str, Any]],
621 ) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
622     """Synthetic nowcast based on the latest available official issue only."""
623     safe_l, safe_v, meta = source
624     issue = int(meta["issue_hour"])
625     offset = (event - issue) * 6
626     if offset < 0:
627         raise ValueError(" 不能使用未来发布的预报")
628     if not offset:
629         return source
630     point = np.asarray(meta["pv_point_kw"])
631     first = max(0, offset - 18)
632     realized = data.actual_pv_kw[day, issue * 6 + first:event * 6]
633     bias = float(np.mean(realized - point[first:offset])) if realized.size else
0.0
634     tau = np.arange(1, N_SLOTS - event * 6 + 1) / 6.0
635     revised = np.maximum(0.0, safe_v[offset:] + bias * np.exp(-tau / 3.0))
636     return safe_l[offset:], revised, {
637         **{k: v for k, v in meta.items() if k not in ("pv_point_kw",
        "bias_kw")},

```

```

638         "decision_hour": event, "synthetic_nowcast": True,
639         "observed_end_exclusive": event * 6, "nowcast_bias_kw": bias,
640     }
641
642
643 def run_q3_day(
644     data: Inputs,
645     day: int,
646     soc0: float,
647     load_point: np.ndarray,
648     load_residual: np.ndarray,
649     pv_issue_residual: np.ndarray,
650     *,
651     variable_price: bool,
652     alpha: float = DEFAULT_ALPHA,
653     safe_updates: list[tuple[np.ndarray, np.ndarray, dict[str, Any]]] | None =
        None,
654     events: tuple[int, ...] = ISSUES,
655     battery: Battery = DEFAULT_BATTERY,
656     emergency_multiplier: float = 5.0,
657     history_days: int = 60,
658     max_scenarios: int = 9,
659 ) -> dict[str, Any]:
660     if not events or events[0] != 0 or tuple(sorted(set(events))) != events or
        any(h < 0 or h >= 24 or int(h) != h for h in events):
661         raise ValueError(" 更新时刻必须从 0 开始, 严格递增且在 [0,24) 内")
662     price = predict_price(data, day) if variable_price else data.typical_price
663     settlement_price = data.variable_price[day] if variable_price else
        data.typical_price
664     if safe_updates is None:
665         safe_l0, safe_v0, meta0 = safe_update_trajectory(
666             data, day, 0, load_point[day], load_residual, pv_issue_residual,
667             alpha=alpha, history_days=history_days, max_scenarios=max_scenarios
668         )
669     else:
670         safe_l0, safe_v0, meta0 = safe_updates[0]
671     initial = solve_schedule(
672         safe_l0,
673         safe_v0,
674         price,
675         soc0,
676         terminal_value=terminal_value(price, battery),
677         battery=battery,
678     )
679     p0 = initial["grid_kwh"]

```

```

679     final_contract = p0.copy()
680     actual_parts: dict[str, list[np.ndarray]] = {
681         key: []
682         for key in (
683             "charge_kwh",
684             "discharge_kwh",
685             "emergency_kwh",
686             "unused_contract_kwh",
687             "pv_curtailed_kwh",
688             "pv_used_kwh",
689             "settlement_cost",
690             "alternative_settlement_cost",
691             "emergency_cost",
692             "cash_cost",
693             "alternative_cash_cost",
694         )
695     }
696     soc_trace = [float(soc0)]
697     update_meta = [meta0]
698     update_solutions: list[dict[str, Any]] = []
699     current_soc = float(soc0)
700
701     for i, issue in enumerate(events):
702         start = issue * 6
703         end = events[i + 1] * 6 if i + 1 < len(events) else N_SLOTS
704         if i == 0:
705             contract_remaining = p0[start:]
706             update_solution = initial
707         else:
708             latest_index = max(j for j, h in enumerate(ISSUES) if h <= issue)
709             if safe_updates is None:
710                 source = safe_update_trajectory(
711                     data, day, latest_index, load_point[day], load_residual,
712                     pv_issue_residual, alpha=alpha, history_days=history_days,
713                     max_scenarios=max_scenarios
714                 )
715             else:
716                 source = safe_updates[latest_index]
717             safe_l, safe_v, meta = nowcast_trajectory(data, day, issue, source)
718             update_meta.append(meta)
719             remaining_price = predict_price(data, day, issue) if variable_price
720             else price[start:]
721             update_solution = solve_schedule(
722                 safe_l,
723                 safe_v,

```

```

721         remaining_price,
722         current_soc,
723         terminal_value=terminal_value(remaining_price, battery),
724         base_plan_kwh=p0[start:],
725         battery=battery,
726     )
727     update_solution["planning_price"] = remaining_price
728     contract_remaining = update_solution["grid_kwh"]
729     update_solutions.append(update_solution)
730     segment_contract = contract_remaining[: end - start]
731     final_contract[start:end] = segment_contract
732     segment = causal_dispatch(
733         segment_contract,
734         data.actual_load_kw[day, start:end],
735         data.actual_pv_kw[day, start:end],
736         settlement_price[start:end],
737         current_soc,
738         base_plan_kwh=p0[start:end],
739         battery=battery,
740         emergency_multiplier=emergency_multiplier,
741     )
742     for key in actual_parts:
743         actual_parts[key].append(segment[key])
744     soc_trace.extend(segment["soc_kwh"][1:].tolist())
745     current_soc = float(segment["soc_kwh"][-1])
746
747     actual = {key: np.concatenate(parts) for key, parts in
748               actual_parts.items()}
749     actual["soc_kwh"] = np.asarray(soc_trace)
750     balance = final_contract - actual["unused_contract_kwh"] +
751               actual["pv_used_kwh"] + actual["discharge_kwh"] + actual["emergency_kwh"] -
752               data.actual_load_kw[day] * DT_HOURS - actual["charge_kwh"]
753     actual["max_balance_residual"] = float(np.max(np.abs(balance)))
754     return {
755         "initial_plan": initial,
756         "initial_plan_kwh": p0,
757         "final_contract_kwh": final_contract,
758         "actual": actual,
759         "update_meta": update_meta,
760         "update_solutions": update_solutions,
761         "planning_price": price,
762         "events": events,
763     }

```

```
763 def summarize_day(result: dict[str, Any], *, adjusted: bool) -> dict[str,
    float]:
764     actual = result["actual"]
765     plan = result["initial_plan_kwh"] if adjusted else
        result["plan"]["grid_kwh"]
766     contract = result["final_contract_kwh"] if adjusted else plan
767     return {
768         "plan_kwh": float(np.sum(plan)),
769         "final_contract_kwh": float(np.sum(contract)),
770         "cash_cost_yuan": float(np.sum(actual["cash_cost"])),
771         "alternative_cash_cost_yuan":
            float(np.sum(actual.get("alternative_cash_cost",
                actual["cash_cost"]))),
772         "emergency_kwh": float(np.sum(actual["emergency_kwh"])),
773         "unused_contract_kwh": float(np.sum(actual["unused_contract_kwh"])),
774         "pv_curtailed_kwh": float(np.sum(actual["pv_curtailed_kwh"])),
775         "charge_kwh": float(np.sum(actual["charge_kwh"])),
776         "discharge_kwh": float(np.sum(actual["discharge_kwh"])),
777         "soc_start_kwh": float(actual["soc_kwh"][0]),
778         "soc_end_kwh": float(actual["soc_kwh"][-1]),
779         "soc_min_kwh": float(np.min(actual["soc_kwh"])),
780         "soc_max_kwh": float(np.max(actual["soc_kwh"])),
781         "max_balance_residual_kwh": float(actual["max_balance_residual"]),
782     }
```